2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3503-2697-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/COMPSAC57700.2023.00281

Uncovering Software Supply Chains Vulnerability: A Review of Attack Vectors, Stakeholders, and Regulatory Frameworks

Nafisa Anjum[1], Nazmus Sakib[1,*], Juanjose Rodriguez-Cardenas[2], Corey Brookins[3], Ava Norouzinia[4], Asia Shavers[2], Miranda Dominguez[4], Marie Nassif[5], and Hossain Shahriar [1]
[1]Department of Information Technology, Kennesaw State University, Marietta, GA, United States
[2]Department of Information Systems and Security, Kennesaw State University, Marietta, GA, United States
[3]Department of Mechanical Engineering, Kennesaw State University, Marietta, GA, United States
[4]Department of Computer Science, Kennesaw State University, Marietta, GA, United States
[5]Department of Electrical and Computer Engineering, Kennesaw State University, Marietta, GA, United States
[*]nsakib1@kennesaw.edu

***Abstract*—The proliferation of cyberattacks in the software supply chain (SSC) domain is a critical concern making them a formidable threat to software security and compromising its integrity and credibility which needs to be seriously acknowledged and investigated. This paper aims to conduct a comprehensive study of the various tactics and techniques employed by cybercriminals in this domain along with a focus on exploring the effect of software supply chain stakeholders' traits, limitations, and actions on the probability of a successful attack. Furthermore, this research also identifies the regulatory tools and protocols administrating software supply chains that support decreasing an organization's proneness to these challenges. Our study adopts a rigorous methodology to investigate the frequency of attacks, current defense techniques, and gaps combined with an overview highlighting where ransomware attacks occur amidst this discussion. The findings will provide valuable insights concerning the recent trends in disrupting the security and efficiency of the software supply chain and offer recommendations to researchers, organizations, and practitioners to remain cautious and proactive in their cybersecurity posture.**

***Index Terms*—Software Supply Chain Attacks; Ransomware Attacks; Security; Stakeholders; Regulatory frameworks.**

I. Introduction

Cyber-attacks on organizations' SSCs continue to be a pressing issue. The 2020 SolarWinds incident disrupted the infrastructure of over 18,000 businesses by planting malicious code within their networks– with devastating effects including data loss and program exploitation [1, 2]. In 2021, a "CodeCov" attack was acknowledged where the attacker accessed source codes using the whipped information of several customers from the CodeCov breach [3]. The following destructive event is entitled -NotPetya, where the Russian military intelligence battered Ukrainian organizations by injecting malware into a software program required to be used by every vendor of Ukraine for tax reporting purposes, ultimately affecting 96% of Ukrainian companies [4]. Ledger, a hardware wallet manufacturing company, was compromised where attackers applied open-source intelligence tools to gain admittance to sensitive ledger records and used the obtained information to send out phishing emails violating users' trust [5]. The growing frequency of cyber-attacks shows the urgent need to invest in research on SSC security, protecting them from legal, financial, and reputational damage.

The escalating interest of cybercriminals in the SSC market has become an alarming concern for organizations around the world, particularly with global e-commerce sales accomplishing $4.5 trillion in 2021 and expecting to make $969 billion in the U.S. at year-end [6]. To strengthen security protocols, the U.S. government has been developing a task force to address SSC risks and also to enhance transparency and accountability in this field [7]. The CHIPS and Science Act of 2022 invests $500 million to promote wireless SSC technology, quantum computing, AI in research initiatives. In addition, Professor Cukier recieved a $9 Million NSF grant– aiming to tackle software security issues around the world [8].

The key reason behind the cumulative amount of research on SSC security and integrity is due to its adoption in various domains ranging from education to government infrastructures. For instance, the Carnegie Mellon Software Engineering Institute has employed SSC projects within its CERT Program to raise awareness and develop methodology associated with such risks [9]. As more and more sectors become contingent on software to run their actions, it has become imperious to identify the various attacks that jeopardize the safety of SSCs, for instance, cybercriminals who target SSCs with ransomware attacks. Unveiling the complexities of software supply chains, this paper analyzes potential vulnerabilities, stakeholders, and regulatory frameworks, addressing following three research questions (RQs).

- ***RQ1:*** What are the most common attack vectors and techniques used by attackers to target the software supply chain, and how do they differ from other types of attacks?
- ***RQ2:*** How do the characteristics and practices of software supply chain participants, including software vendors, developers, and users, affect the risk of software supply chain attacks?
- ***RQ3:*** What are the legal and regulatory frameworks governing the software supply chain, and how do they address the risks posed by cyber threat actors? Where do the ransomware attacks take place in this debate?

979-8-3503-2697-0/23/$31.00 ©2023 IEEE
DOI 10.1109/COMPSAC57700.2023.00281

Authorized licensed use limited to: Kennesaw State University. Downloaded on July 28,2025 at 19:50:42 UTC from IEEE Xplore. Restrictions apply.

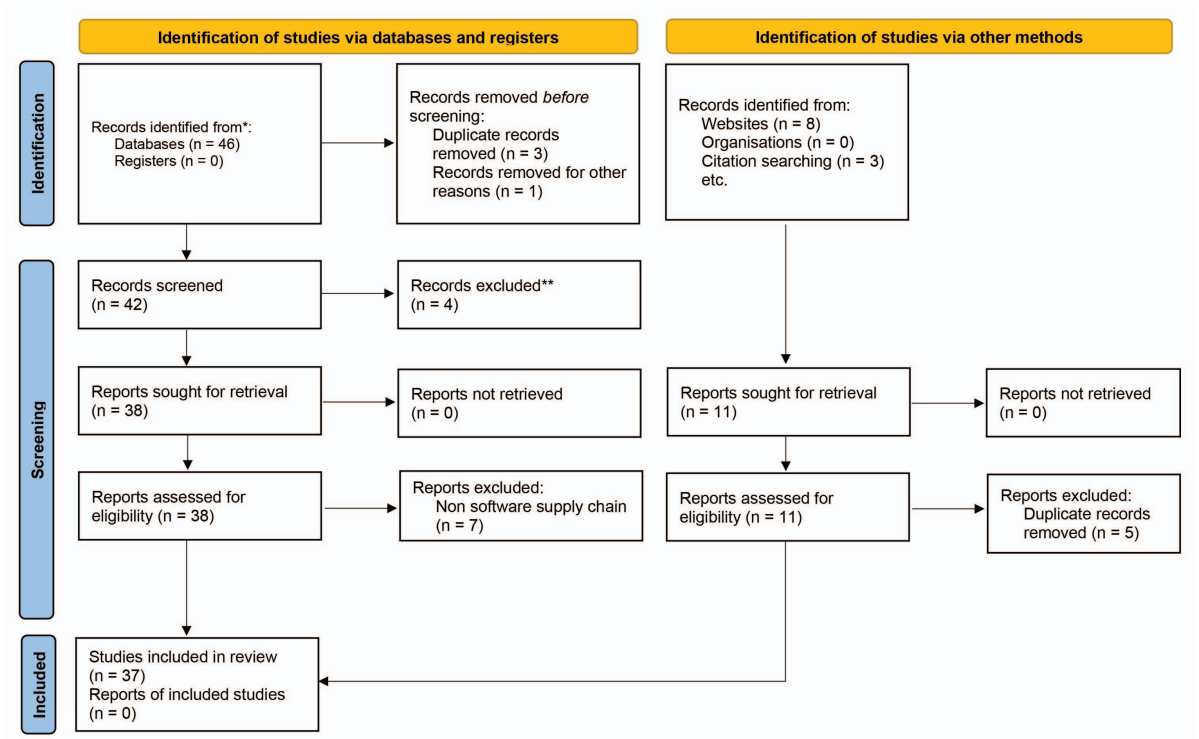

Fig. 1: PRISMA flow diagram illustrating the process of systematic literature search

The paper is structured as follows to provide a clear overview. Section II presents a systematic method for collecting relevant scientific papers and gleaning their information and insights. Then, section III will portray an overview of malicious cyber-attacks, existing solutions, and future opportunities. Finally, we wrap up with a summary highlighting our key discoveries and conclusions.

II. Methodology

This study is structured to give an in-depth analysis and response to carefully chosen research questions, utilizing a distinctly outlined methodology that includes detailed scoping criteria and systematic data collection.

A. Scoping Criteria

The scoping criteria for understanding a range of solution techniques safeguarding SSCs refer to the significant rise in the occurrence of specific attacks within a selected timeframe, such as in the last five years. The growing emphasis on SSC and the increasing rate of third-party elements employed in software development have recognized them as a tempting target for a broad range of high-profile attacks, including Ransomware attacks, SolarWind Attacks [10], Superfish Attacks [11], Trojan Horse or Trojan [12] and so on. To ensure relevance, we have documented literature based on the latest menaces in SSCs of our research and developed a preliminary understanding of productive solutions such as cryptographic techniques, code signing, and so forth that help to safeguard the integrity of software applications.

B. Systematic Literature Search

Another vital component for conducting this methodology was executing a systematic literature review within the sphere of the SSC through a straightforward process that facilitated identifying admissible literature for insertion in the survey paper. A wide range of academic databases was determined, specifically Google Scholar, IEEE Xplore, and Elsevier. Relevant keywords and specific phrases derived from the research questions were then articulated, consisting of "SSC attacks", "The risk of ransomware attacks in SSC", "frameworks governing SSC", "practices of SSC participants", "practical experiences with SSC security", "SSC security and machine learning," "open-source software," "latest technologies for SSC security" to offer a solid foundation for our findings and conclusions. The inclusion criteria also depended on solutions provided in included papers and the period in which each paper was published. The exclusion of irrelevant articles was applied through a screening process of titles and abstracts. Ultimately, we identified 37 articles using a PRISMA flowchart that satisfied all selection criteria shown in Figure 1. This visual interpretation will perform as a significant tool in delivering a clear documentation of the inclusion and exclusion criteria, reducing bias, and confirming the comprehensibility and methodological vigor of the study.

C. Data Analysis Procedure

The data analysis procedure involves summarizing the outcomes derived from selected literature that explicitly addresses our research questions. We conducted a systematized and

Authorized licensed use limited to: Kennesaw State University. Downloaded on July 28,2025 at 19:50:42 UTC from IEEE Xplore. Restrictions apply.

integrated qualitative analysis to examine specific categories such as the structure of the latest and most threatening SSC attacks, how these attacks occur, the countermeasures against SSC attacks, exploitation of SSC design by software participants, examples of administrative protocols in real-world circumstances and efficacy level based on the documentation of crucial facts across the literature. Additionally, this analysis will assist in the in-depth evaluation of SSC outbreaks and results, ensuring the validity, scalability, and reliability of this survey paper's outcomes to the selected audience.

III. Results

This section presents the results of our literature review which address three research questions. We give a brief overview of the software supply chain and discuss catastrophic scenarios that could lead to failure throughout its development life cycle (SDLC). Next, we look at security measures put in place to counter SSC attacks as well as how ransomware exacerbates possible threats.

A. Understanding the Attack Vectors targeting Software Supply Chain (RQ1)

The rise of software supply chain attacks has a far-reaching impact than any other targeted attacks as it has the potential to affect multiple organizations with just one compromised software component, producing more than 50% of overall cybersecurity attacks [13]. In most cases, these attacks can be characterized as inserting malicious code targeting the downstream links [14, 15]. On the other hand, various studies depict that such attacks can also take forms other than strict code injection. By scrutinizing the theory of SSC attacks from several resources, it becomes apparent that these attacks follow a four-stage pattern: first, a cyber threat actor identifies and compromises a prevailing vulnerability within the system, such as in the product's code, or software dependencies; second, they control the vulnerability to manipulate the SSC; third, the modification created disseminates to downstream elements; and finally, the changes are exploited by the attacker in a downstream link to disrupt the security, integrity, and validity of a particular software product [15, 16]. Malicious software operators leverage a multitude of techniques, tactics, and procedures for manipulating a software's lifecycle to destabilize software practices and controls effectively:

1) Malicious Commits: Malicious commits occur when a malevolent actor secretly modifies a code in a publicly accessible repository, making it appear harmless [3]. An instance of this was seen when malicious code was added by a hacker who disguised their addition as a typographical fix under the designation of the inventor of PHP, facilitating backdoor entry to websites running the counterfeit version of the code.

2) Poisoning with Backdoor: The implementation of backdoors enables an unauthorized user to avoid traditional access techniques via code or configurations and acquire entry to a computer device or server without the need to have information of usernames or passwords [17]. For example, a code-based backdoor may utilize custom administration interfaces that are instigated in the base code by maneuvering the authentication procedure to observe explicit hard-coded hashes and permitting access regardless of the authentic users delineated by the administrator.

3) Domain Name Exploitation (Typosquatting): Cybercriminals have formulated another technique known as "typosquatting" to manipulate domain names by registering them with typographical alternatives of targeted domains [18]. A malicious actor establishes his package confining harmful payloads under designations similar to the genuine packages. Due to the substantial number of owners and managers, these attacks can go unobserved for a significant time [3].

4) Unauthorized access to Update Servers: Ensuring timely installation of the newest updates is regarded as the best practice, typically accomplished through an automating procedure with the help of update servers. However, in some cases, software updates can be leveraged to introduce malicious code by placing their payload inside update packages before they are cryptographically signed off and then delivering it to the target organization.

5) Downloading Trouble through App Stores: App stores offer a central platform that simplifies the search for software, improving the consumer experience. However, these high-traffic platforms relating customers with third-party developers have become a prevalent attack target on the SSC. Attackers frequently exploit user trust by camouflaging malicious software with a legitimate appearance, offering repackaged apps containing malware on third-party sites, and injecting malware into app development tools before release. Through this deception, they can spread their invasive codes around the globe [19].

6) Software Supply Chain Attacks using Infection Vectors: To target a sector-specific organization, utilization of infection vectors or, in simple terms, the combination of various individual methods can offer access to operational facilities to a point where attackers could potentially control everything at their discretion. A sector-specific attempt was Dragonfly 2.0, where various companies worldwide belonging to the energy sector were targeted using a variety of infection vectors to infiltrate organizations' networks [20].

Table 1 represents some practical examples of SSC attacks implemented using techniques mentioned earlier, resulting in devastating consequences.

B. Assessing the impact of software supply chain participants' behavior on the security risk (RQ2)

To restate the above-mentioned point, a malicious entity can compromise the SSC security by penetrating the network of a software vendor using maleficent code before it is supplied to the end users. However, the integrity of SSCs are also affected by the distinguishable traits and customs of the stakeholders, such as the software developers, distributors, and consumers involved in various layers of software operations. At the very bottom of the line comes the role of software developers who sometimes may choose to avoid updating crucial dependencies due to two key factors: the inadequacy of resources required to conduct appropriate dependency management and potential of

Authorized licensed use limited to: Kennesaw State University. Downloaded on July 28,2025 at 19:50:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I: A comprehensive overview of real-world software supply chain attack scenarios

Incident Name	Year	Technique Applied	Description	References
Backdoor positioned on PHP Git Repository	2021	Malicious Commits through Code-Signing in PHP Git Repository	The base code of the formal PHP Git server was compromised by threat actors who utilized code-signing and took the identity of renowned PHP developers, then finally inserted two malicious commits that secretly injected a backdoor allowing remote code execution.	[21]
Octopus Scanner Attack	2020	Insertion of backdoor malware in open-source SSC	A total of 26 open-source NetBeans projects backdoored by malicious contents within both project and JAR files and started spreading the compromised code throughout the repositories by itself while the owners were absolutely oblivious of the ongoing activities.	[22]
js-sha3 campaign	2019	Typosquatting to import malicious packages	25 packages with hostile contents were identified by npm security team and Microsoft Vulnerability Research team where the attacker replaced one package named js-sxa3 with a harmful package js-sha3. It was highly doubtful for a developer to restitute "h" with an "x" due to their gap on a keyboard.	[14], [23]
Kingslayer Attack	2017	Credential theft and compromised update application by insertion of "Trojan" horse	Attackers targeted system administrator accounts to obtain login credentials and then exchanged original update application with a malicious one containing a "backdoor" to grant constant access, retrieve sensitive data, and upload extra malware. Among the entities targeted were several windows operating system administrators such as– 5 chief defense contractors; 4 telecommunications providers, 10 western military organizations, and minimum 45 educational institutions.	[24], [25]
Pokémon Go Attack	2016	Malware Infested App for Download	Attackers repackaged Android editions of Pokémon and appended their own DroidJack malware, disseminating the infected software to users as a downloadable application on third-party stores; once downloaded it had the capability to exfiltrate all data of the compromised device by asking for additional app authorizations.	[19]
Dragonfly 2.0	2015	Insertion of a selection of infection vectors for data extraction in the Energy sector	Threat actors obtained delicate login credentials from organizations in the U.S., Turkey, and Switzerland by sending malicious emails, deploying watering hole attacks, and employing a trojan software at the same time and successfully achieved the capacity to disrupt the systems completely.	[20]

new dependency forms in introducing breaking changes which are often problematic to manage [26]. Several developers also prefer to count on social platforms for vulnerability and functionality solutions and get dazed by the massive amount of redundant information rather than using dependency analysis tools that can identify crucial security problems in the software projects [27]. Furthermore, in some instances, developers compromise the cryptographic hashes by storing them with the software packages in the same distribution site, presenting an opportunity for attackers to effortlessly replace or modify the software with a damaged version [28].

To further elaborate on this topic, both software vendors and end users play substantial parts in guaranteeing the security of SSC. The incautious nature of vendors regarding the distribution of routine updates could easily be confiscated by attackers who may implant malicious code to rupture the user network [29]. The majority part of the users generally do not vigorously examine code signatures and rely solely on the trustworthiness of the vendor, making the system more sensitive by allowing cybercriminals to sign in as a trusted vendor without the knowledge of the actual owner. However, the mentioned measures can be acquired not only through the activities of threat actors but also from mistakes of legitimate authorities, insider attackers, or resellers with less thorough scrutiny. The impulsive behavior of vendors and users while copying and pasting codes for open-source software may lead to the unintended introduction of exposures into the system [30]. Vendors could also opt to use outdated software versions containing vulnerabilities rather than a secured and updated version making it defenseless to misuse and exploitation [22]. Figure 2 illustrates the actions of software supply chain stakeholders in affecting the exposure to SSC attacks throughout various phases of the SDLC.

The SDLC is composed of three distinct communities of practice: the developer, the supplier, and the customer- each commanding a defined process to accomplish their respective responsibilities. The likelihood of disrupting the entire system may occur due to innumerable activities performed by any of these communities, including the insertion of malicious updates, unawareness of process management, supplying counterfeit applications, or even incompetence. Therefore, it has become imperative to develop a set of strategies or policies to ensure the security of the global software supply chain.

C. Regulatory Frameworks Addressing Software Supply Chain Attacks (RQ3)

The legal frameworks governing the domain of software supply chains are still in the developing stage. However, they offer an outstanding possibility to ensure the safety, transparency, and authenticity of the software products, crucial for all stakeholders engaged. This section will explore some of the existing preventive measures for decreasing the vulnerabilities that could have been expressively or unknowingly injected into any SSC. Most malicious open-source packages introduce security risks to SSC by adding a substantial amount of new artifacts during installation instead of benign versions of the same package. To address this issue, researchers have proposed Buildwatch framework for performing dynamic analysis of software and its third-party dependencies by mining the observables, such as noise or outliers, computing the difference from former behavior and removing them before causing any

Authorized licensed use limited to: Kennesaw State University. Downloaded on July 28,2025 at 19:50:42 UTC from IEEE Xplore. Restrictions apply.

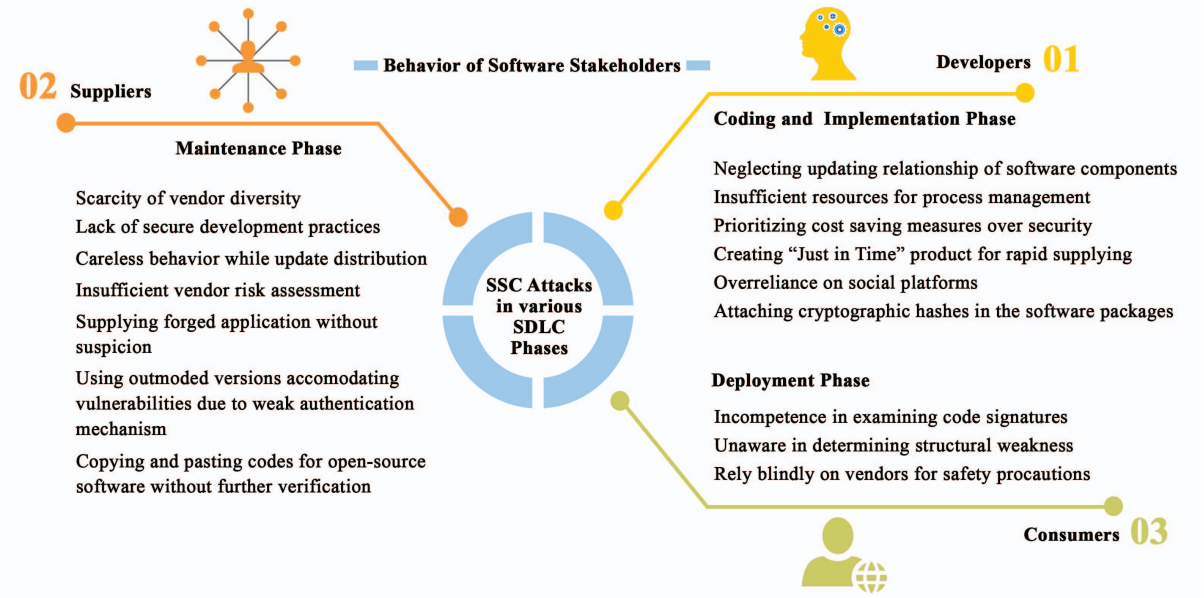

Fig. 2: Characteristics and challenges modeled by software supply chain participants

damage to the build systems [31]. Studies have also suggested utilizing an information flow-based framework that enables a user to identify a range of SSC attacks without having to communicate with any suppliers and without requiring any previous knowledge in this area. In the traffic anomaly detection technique, normal traffic flow has been used as a reference. The detection system can easily categorize attacks by analyzing the unseen data exfiltration and Command and Control traffic that solitarily appears during an attack [32]. A cybercriminal can also compromise any layer in the SDLC ranging from the development to deployment phase and illegally modify the software product. To mitigate this problem, an "in-toto" framework in collaboration with open-source projects has been recommended to safeguard the authenticity and safety of cloud-native, hybrid-cloud, and cloud-agnostic applications that consent participants to the ability to validate their SSC [33]. Without proper regulatory frameworks, the likelihood of an unfavorable breach compromising the global SSC is impending. The presence of increasingly sophisticated cyber threats calls for adaptive, recurrently updated frameworks to effectively evaluate and mitigate risks to stay ahead of the ever-evolving security landscape.

IV. Intersection of Software Supply Chain and Ransomware Attacks

The services provided by SSCs have received the attention of many cybercriminals and revealed a considerable collection of disturbing incidents, most of which belong to the domain of ransomware threats. In July 2021, Kaseya's Virtual System Administrator (VSA) platform was infected by a highly developed zero-day vulnerability SSC attack compromising over 1500 companies and decrypting thousands of customers' valuable records. Attackers uploaded a random file injected with malicious code into the VSA software while circumventing all security protocols allowing them to establish ransomware on every accessible system [34]. The next noteworthy ransomware software supply chain attack is the famous SolarWinds attack, which had enormous implications for governmental and commercial institutions. SUNBURST malware was secretly implanted into an update package that victims unknowingly installed onto their SolarWinds system and allowed attackers to add more malicious payloads without raising suspicion [35]. The NotPetya attack executed in 2017, attacks against Colonial Pipeline and JBS are some other examples of the most significant and destructive ransomware software supply chain attacks in history [3].

V. Findings & Conclusion

Our findings, derived from Figure 2, suggest that adversaries' sophisticated tactics don't only determine the security of SSCs. Instead, it requires assessment at every stage during the development process- including design and testing to distribution. This highlights a crucial gap in research regarding interactions between System Development Life Cycle (SDLC) and SSC Management systems (SSCM), hinting these may have solutions for pressing security issues concerning SSCs. The Software Development Life Cycle (SDLC) is an essential framework for the successful creation of software, from concept to delivery. It takes control over each element involved in the development and enforces discipline through automated testing and versioning systems, allowing stakeholders to monitor any modifications made during production. SSCM framework ensures that an independent body has reviewed all aspects of a product's journey from creation to delivery and complies with necessary regulations. Further exploring avenues for collaboration between these frameworks can help organizations, vendors, and customers effectively manage their SSC while protecting against internal and external threats.

Authorized licensed use limited to: Kennesaw State University. Downloaded on July 28,2025 at 19:50:42 UTC from IEEE Xplore. Restrictions apply.

SSC attacks are a growing threat, with malicious actors able to commit financial theft, disable networks and gain access to sensitive data. In this study, we identified the tactics employed by cybercriminals in SSCs as well as outlined frameworks used for mitigating harm. By looking at these approaches, it is possible to control or even prevent such dangers from occurring. Our research uncovered how attackers take advantage of practitioners' actions across all stages of the SDLC. Our future research direction— grounded on these research findings— will focus on developing more aware stakeholders within the SDLC ecosystem.

VI. Acknowledgements

The work is partially supported by Kennesaw State University (KSU) Office of Undergraduate Research Award and the National Science Foundation (NSF) under NSF award #2100115 and #2209638. Any opinions, findings, recommendations, expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

[1] S. Benthall, "Assessing software supply chain risk using public data," in *2017 IEEE 28th Annual Software Technology Conference (STC)*. IEEE, 2017, pp. 1–5.

[2] M. Willett, "Lessons of the solarwinds hack," *Survival*, vol. 63, no. 2, pp. 7–26, 2021.

[3] S. Cordey, "Software supply chain attacks: An illustrated typological review," *CSS Cyberdefense Reports*, 2022.

[4] M. Crosignani, M. Macchiavelli, and A. F. Silva, "Pirates without borders: The propagation of cyberattacks through firms' supply chains," *Journal of Financial Economics*, vol. 147, no. 2, pp. 432–448, 2023.

[5] U. Agarwal, V. Rishiwal, S. Tanwar, R. Chaudhary, G. Sharma, P. N. Bokoro, and R. Sharma, "Blockchain technology for secure supply chain management: A comprehensive review," *IEEE Access*, 2022.

[6] S. Intelligence. (Year of publication, Month of publication) Global supply chain security threats: How to handle. Security Intelligence. [Online]. Available: https://securityintelligence.com/articles/global-supply-chain-security-threats-how-to-handle/

[7] J. Boyens, C. Paulsen, N. Bartol, K. Winkler, and J. Gimbi, "Key practices in cyber supply chain risk management: Observations from industry," National Institute of Standards and Technology, Tech. Rep., 2021.

[8] C. o. E. University of Maryland. (Year of publication, Month of publication) Nsf awards grant to support supply chain security research. University of Maryland, College of Engineering. [Online]. Available: https://eng.umd.edu/news/story/nsf-awards-grant-to-support-supply-chain-security-research

[9] C. J. Alberts, A. J. Dorofee, R. Creel, R. J. Ellison, and C. Woody, "A systemic approach for assessing software supply-chain risk," in *2011 44th Hawaii International Conference on System Sciences*. IEEE, 2011, pp. 1–8.

[10] R. Alkhadra, J. Abuzaid, M. AlShammari, and N. Mohammad, "Solar winds hack: In-depth analysis and countermeasures," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, 2021, pp. 1–7.

[11] J. Reuben and N. Ware, "Approach to handling cyber security risks in supply chain of defence sector," *Industrial Engineering Journal*, vol. 12, no. 7, pp. 1–12, 2019.

[12] K. R. Ludvigsen, S. Nagaraja, and A. Daly, "Preventing or mitigating adversarial supply chain attacks: A legal analysis," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 2022, pp. 25–34.

[13] M. J. H. Faruk, M. Tasnim, H. Shahriar, M. Valero, A. Rahman, and F. Wu, "Investigating novel approaches to defend software supply chain attacks," in *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2022, pp. 283–288.

[14] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "Taxonomy of attacks on open-source software supply chains," *arXiv preprint arXiv:2204.04008*, 2022.

[15] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, "Sok: Analysis of software supply chain security by establishing secure design properties," 2022.

[16] S. Simpson, D. Baldini, G. Bits, D. Dillard, C. Fagan, B. Minnis, and D. Reddy, "Software integrity controls. an assurance-based approach to minimizing risks in the software supply chain," *SAFECode, June*, 2010.

[17] M. Agrawal, H. Singh, N. Gour, and M. A. Kumar, "Evaluation on malware analysis," *International Journal of Computer Science and Information Technologies*, vol. 5, no. 3, pp. 3381–3383, 2014.

[18] J. Spaulding, S. Upadhyaya, and A. Mohaisen, "The landscape of domain name typosquatting: Techniques and countermeasures," in *2016 11th International Conference on Availability, Reliability and Security (ARES)*. IEEE, 2016, pp. 284–289.

[19] Atlantic Council. (2020, July 26) Breaking trust: Shades of crisis across an insecure software supply chain. Atlantic Council. [Online]. Available: https://www.atlanticcouncil.org/in-depth-research-reports/report/breaking-trust-shades-of-crisis-across-an-insecure-software-supply-chain/

[20] S. E. Blogs. (Year of publication, Month of publication) Dragonfly: Western energy sector targeted by sophisticated cyber attacks. Symantec Enterprise Blogs. [Online]. Available: https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence/dragonfly-energy-sector-cyber-attacks

[21] BleepingComputer. (Year of publication, Month of publication) Php's git server hacked to add backdoors to php source code. BleepingComputer. [Online]. Available: https://www.bleepingcomputer.com/news/security/phps-git-server-hacked-to-add-backdoors-to-php-source-code/

[22] D. Yan, Y. Niu, K. Liu, Z. Liu, Z. Liu, and T. F. Bissyandé, "Estimating the attack surface from residual vulnerabilities in open source software supply chain," in *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2021, pp. 493–502.

[23] M. Taylor, R. K. Vaidya, D. Davidson, L. De Carli, and V. Rastogi, "Spellbound: Defending against package typosquatting," *arXiv preprint arXiv:2003.03471*, 2020.

[24] C. Clancy, J. Ferraro, R. Martin, A. Pennington, C. Sledjeski, and C. Wiener, "Deliver uncompromised: Securing critical software supply chains," *MITRE Technical Papers*, vol. 24, p. 01, 2021.

[25] D. Blessman, "Protecting your software supply chain," *Risk Management*, vol. 66, no. 1, pp. 10–11, 2019.

[26] D. Gonzalez, T. Zimmermann, P. Godefroid, and M. Schäfer, "Anomalicious: Automated detection of anomalous and potentially malicious commits on github," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 258–267.

[27] P. Goswami, S. Gupta, Z. Li, N. Meng, and D. Yao, "Investigating the reproducibility of npm packages," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 677–681.

[28] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, 2022, pp. 331–340.

[29] C. W. Lee and G. W. Ulferts, "Managing supply chain risks and risk mitigation strategies," *North Korean Review*, pp. 34–44, 2011.

[30] Cybersecurity and Infrastructure Security Agency, "Defending against software supply chain attacks," Technical Report, April 2021, publication.

[31] M. Ohm, A. Sykosch, and M. Meier, "Towards detection of software supply chain attacks by forensic artifacts," in *Proceedings of the 15th international conference on availability, reliability and security*, 2020, pp. 1–6.

[32] X. Wang, "On the feasibility of detecting software supply chain attacks," in *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)*. IEEE, 2021, pp. 458–463.

[33] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes." in *USENIX Security Symposium*, 2019, pp. 1393–1410.

[34] J. Martínez and J. M. Durán, "Software supply chain attacks, a threat to global cybersecurity: Solarwinds' case study," *International Journal of Safety and Security Engineering, vol*, vol. 11, no. 5, pp. 537–545, 2021.

[35] E. D. Wolff, K. Growley, M. Gruden *et al.*, "Navigating the solarwinds supply chain attack," *The Procurement Lawyer*, vol. 56, no. 2, 2021.

Authorized licensed use limited to: Kennesaw State University. Downloaded on July 28,2025 at 19:50:42 UTC from IEEE Xplore. Restrictions apply.